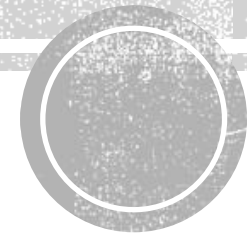




python

Apprendre
à coder
avec



Pr. Houda BENHAR

PLAN

- Expressions booléennes
- Les structures conditionnelles
- Gestion des exceptions
- Structures itératives: boucles for et while
- Les fonctions et modules



EXPRESSIONS BOOLÉENNES

- Une expression booléenne est une expression qui est vraie ou fausse.
- Les exemples suivant utilisent l'opérateur `==`, qui compare deux opérandes et produit `True` si ils sont égaux et `False` sinon:

```
>>> 5 == 5
True
>>> 5 == 6
False
```

- `True` et `False` sont des valeurs spéciales qui appartiennent à la classe `bool` ; ils ne sont pas des strings:

```
>>> type(True)
<class 'bool'>
>>> type(False)
<class 'bool'>
```



EXPRESSIONS BOOLÉENNES

Opérateurs de comparaison

- L'opérateur `==` est l'un des opérateurs de comparaison; les autres sont:

<code>x != y</code>	<code># x is not equal to y</code>
<code>x > y</code>	<code># x is greater than y</code>
<code>x < y</code>	<code># x is less than y</code>
<code>x >= y</code>	<code># x is greater than or equal to y</code>
<code>x <= y</code>	<code># x is less than or equal to y</code>
<code>x is y</code>	<code># x is the same as y</code>
<code>x is not y</code>	<code># x is not the same as y</code>

- Exemples:

```
>>> 1 + 2 < 3.4
True
>>> 5.6 >= 7
False
>>> "abc" != "def"
True
```



EXPRESSIONS BOOLÉENNES

Opérateurs logiques

- Il existe trois opérateurs logiques: **and**, **or**, et **not**. La sémantique de ces opérateurs est similaire à leur signification en anglais.
- Exemples:

```
>>> not False
True
>>> 1 + 2 < 3 and 4.5 == 6
False
>>> 7 <= 8 or "abc" == "def"
True
>>> x = 12
>>> y = 12.5
>>> not (x > y)
True
```



STRUCTURES CONDITIONNELLES - IF

- Ça nous permet d'exécuter certaines commandes ou certaines lignes de notre code seulement si certaines conditions sont vérifiées.
- La forme la plus simple est l'instruction *if*:

```
>>> a = 33
>>> b = 200
>>> if b > a:
    print("b is greater than a")
```

```
b is greater than a
```



STRUCTURES CONDITIONNELLES – IF.ELSE

- Une deuxième forme de l'instruction *if* est l'exécution alternative *if..else*:

```
>>> if a % 2 == 0 :  
        print('a is even')  
else:  
        print('a is odd')
```

```
a is odd
```

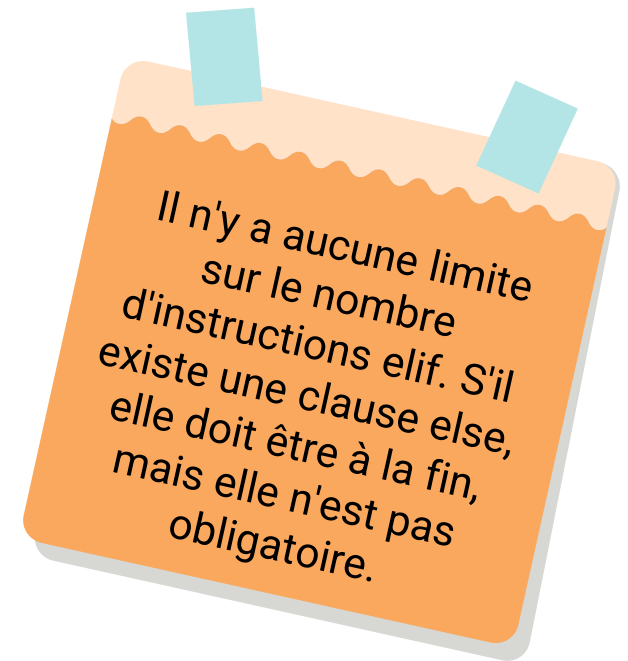


STRUCTURES CONDITIONNELLES – IF..ELIF..ELSE

- La structure If..elif..else en Python est utilisée pour écrire des conditions enchaînées:

```
>>> a = 10
>>> b = 10
>>> if a > b:
    print("a is greater than b")
elif a < b:
    print("b is greater than a")
else:
    print("a and b must be equal")
```

a and b must be equal



STRUCTURES CONDITIONNELLES - BONUS

- Parfois, il est utile d'avoir un corps sans instructions (généralement, pour réserver de la place pour du code que vous n'avez pas encore écrit). Dans ce cas, vous pouvez utiliser l'instruction **pass**, qui ne fait rien:

```
>>> if a > 0:
    pass #TODO: need to handle positive values!
```

- Si vous n'avez qu'une seule instruction à exécuter, vous pouvez la placer sur la même ligne que l'instruction **if**:

```
>>> if a > b: print("a is greater than b")
```

- Pour une instruction **if..else**, Si vous n'avez qu'une seule instruction à exécuter pour **if**, et qu'une seule pour **else** vous pouvez tout mettre sur la même ligne:

```
>>> print("A") if a > b else print("B")
```



STRUCTURES CONDITIONNELLES — EXERCICE

- Que va afficher le code suivant si $x = 3$, $y = 5$, et $z = 2$?

```
if x < y and x < z:  
    print("a")  
elif y < x and y < z:  
    print("b")  
else:  
    print("c")
```



LES EXCEPTIONS

Les exceptions sont des événements qui se produisent lors de l'exécution d'un programme et qui perturbent le flux normal d'exécution.

Python fournit de nombreux types d'exceptions prédéfinies, comme **SyntaxError**, **TypeError**, **ValueError**, **ZeroDivisionError**, etc.

```
>>> 4 + spam*3
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    4 + spam*3
NameError: name 'spam' is not defined
>>> '2' + 2
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    '2' + 2
TypeError: can only concatenate str (not "int") to str
>>> 10 * (1/0)
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    10 * (1/0)
ZeroDivisionError: division by zero
```

Les exceptions ne sont pas toujours fatales et peuvent être traitées



LES EXCEPTIONS

Les clauses **try** et **except** permettent de tester (try) un code qui peut potentiellement poser problème et de définir les actions à prendre si une exception est effectivement rencontrée (except):

Syntaxe 1:

```
try:  
    #code susceptible de renvoyer une erreur  
except:  
    #code à exécuter si une exception survient
```

Syntaxe 2:

```
try:  
    #code  
except Exception:  
    #code
```



```
TypeError  
NameError  
ValueError  
ZeroDivisionError  
...
```



LES EXCEPTIONS

Exemple 1:

```
>>> try:
    x = int(input("Entrez un nombre entier : "))
except ValueError:
    print("Vous devez entrer un nombre entier valide.")
```

Exemple 2:

```
>>> try:
    age = int(input("Entrez votre âge : "))
    ratio = 10 / age
except:
    print("Attention! Une erreur est survenue")
```



LES EXCEPTIONS

Exemple 3:

```
>>> try:
    age = int(input("Entrez votre âge : "))
    ratio = 10 / age
except ValueError:
    print("Vous devez entrer un nombre entier pour votre âge.")
except ZeroDivisionError:
    print("Vous ne pouvez pas diviser par zéro.")
```

Exemple 4:

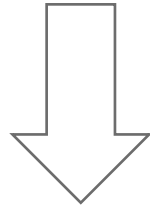
```
>>> try:
    age = int(input("Entrez votre âge : "))
    ratio = 10 / age
except (ValueError, ZeroDivisionError):
    print("Attention! Une erreur est survenue")
finally:
    print("Fin")
```



LES EXCEPTIONS

Exemple 5:

```
fahr = float(input('Enter Fahrenheit Temperature: '))  
cel = (fahr - 32.0) * 5.0 / 9.0  
print(cel)
```



```
try:  
    fahr = float(input('Enter Fahrenheit Temperature: ')) # peut  
    generer une exception  
    cel = (fahr - 32.0) * 5.0 / 9.0  
    print(cel)  
except :  
    print('Une erreur est survenue')
```



EXERCICE

- Écrivez un programme pour demander à l'utilisateur de saisir une note comprise entre 0 et 20. Si la note est hors limites, imprimez un message d'erreur. Si la note est entre 0 et 20, affichez la mention en utilisant le tableau suivant:

≥ 18	Félicitations
≥ 16	Très bien
≥ 14	Bien
≥ 12	Assez bien
≥ 10	Passable
< 10	Inssufisant

Exécutez le programme à plusieurs reprises comme indiqué ci-dessous pour tester les différentes valeurs:

Entrer une note: 20

Félicitations

Entrer une note: 21

Mauvaise note

Entrer une note: 5

Insuffisant

Entrer une note: 14

Bien



LOOPS

- Les boucles s'utilisent pour répéter plusieurs fois l'exécution d'une partie du programme.

Boucle bornée: quand on sait combien de fois doit avoir lieu la répétition, on utilise généralement une boucle for.

Boucle non bornée: si on ne connaît pas à l'avance le nombre de répétitions, on choisit une boucle while.

- Syntaxe:

Boucle while

```
while condition:  
    instructions
```

Exécuter un ensemble d'instructions tant qu'une condition est vraie.

Boucle for

```
for element in collection:  
    instructions
```

Utilisée pour effectuer une itération sur une collection d'objets: liste, tuple, dictionnaire, string.



LOOPS

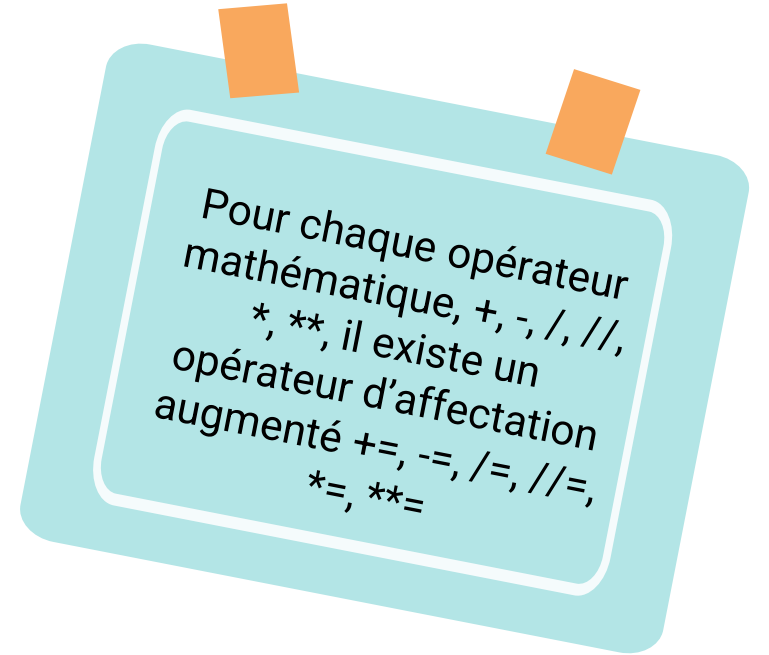
- Exemples d'utilisation:

Boucle while

```
x = 1
while x < 10:
    print("x a pour valeur", x)
    x *= 2
print("Fin")
```

Boucle for

```
name_list = ["toto", "tata", "titi"]
for nom in name_list :
    print(nom)
```



FOR LOOP — RANGE AND LEN FUNCTIONS

- **Fonction len():** renvoie le nombre des éléments (ou la longueur) dans un objet

```
print(len([1, 2, 3])) # liste de 3 éléments
print(len('Python')) # chaîne de 6 caractères
```

- **Fonction range([start], stop, [step]):** retourne une liste de nombres en prenant de 1 a 3 entiers:

début (facultatif): un nombre entier spécifiant à quelle position commencer. La valeur par défaut est 0

stop: un nombre entier spécifiant à quelle position se terminer

étape (facultatif): un nombre entier spécifiant l'incrément. La valeur par défaut est 1

Syntaxe: for **variable_name** in **range(start, stop, step)**

```
for i in range(4):
    print("i a pour valeur", i)
```



BREAK AND CONTINUE

- L'instruction `break` permet de « casser » l'exécution d'une boucle (`while` ou `for`). Elle fait sortir de la boucle et passer à l'instruction suivante.
- L'instruction `continue` permet de passer prématurément au tour de boucle suivant. Elle fait continuer sur la prochaine itération de la boucle

Exemple

```
while True:
    line = input('> ')
    if line[0] == '#':
        continue
    if line == 'done':
        break
    print(line)
print('Done!')
```



LOOPS - EXERCICE

- En utilisant la fonction `range()` essayez d'écrire un programme qui permet de créer et afficher des données coordonnées comme suit:

(0,0)

(0,1)

(1,0)

(1,1)

```
for x in range(2):  
    for y in range(2):  
        print(f"({x}, {y})")
```



LOOPS - EXERCICE

- Écrivez un programme qui lit plusieurs fois des nombres jusqu'à ce que l'utilisateur saisisse "done". Une fois que "done" est entrée, imprimez la moyenne des nombres. Si l'utilisateur saisit autre chose qu'un nombre, détectez leur erreur en utilisant try and except et imprimez un message d'erreur et passer au numéro suivant.

```
total = 0
count = 0
while (True):
    inp = input('Enter a number: ')
    if inp == 'done':
        break
    try:
        value = float(inp)
    except:
        print('Invalid input')
        continue
    total = total + value
    count = count + 1

average = total / count
print('Average:', average)
```



POURQUOI LES FONCTIONS?

- Les fonctions peuvent rendre un programme plus **petit** en **éliminant le code répétitif**. Plus tard, si vous faites un changement, vous n'avez qu'à le faire à un seul endroit.
- Diviser un programme long en fonctions permet de **déboguer** les parties une par une, puis les assembler en un programme **fonctionnel**.
- Des fonctions bien conçues sont souvent utiles pour de nombreux programmes. Une fois que vous écrivez et déboguer une, vous pouvez **la réutiliser**.



FONCTIONS

- Une fonction est une séquence nommée d'instructions qui effectue un calcul ou une tâche spécifique.

Exemple d'appel de fonction:

```
type("chaîne")
```

- Python fournit un certain nombre de fonctions intégrées (**built-in functions**) importantes que nous pouvons utiliser sans avoir à fournir la définition de la fonction: `int()`, `float()`, `str()`, `input()`, `print()`, `len()`, `range()`, `abs()`, `min()`, `max()`, `round()`...



CREER DES FONCTIONS

mot clé qui indique qu'il s'agit d'une définition de fonction.

nom de la fonction

l'en-tête (header) `def print_lyrics():`

```
    print("Emmenez-moi au bout de la terre")  
    print("Emmenez-moi au pays des merveilles")
```

} corps (body) de la fonction

- Les règles pour les noms de fonctions sont les mêmes que pour les noms de variables.
- Eviter d'avoir une variable et une fonction avec le même nom.
- La syntaxe pour appeler la nouvelle fonction est la même que pour les fonctions intégrées :

`print_lyrics()`



CREER DES FONCTIONS

- Une fois que vous avez défini une fonction, vous pouvez l'utiliser dans une autre fonction.

```
def print_lyrics():  
    print("Emmenez-moi au bout de la terre")  
    print("Emmenez-moi au pays des merveilles")
```

```
def repeat_lyrics():  
    print_lyrics()  
    print_lyrics()
```

```
repeat_lyrics()
```

i La définition de la fonction doit être exécutée avant la première fois qu'elle est appelé.



FONCTIONS - EXERCICE

- Que va imprimer le programme Python suivant:

```
def fred():  
    print("Fred")  
def jane():  
    print("ABC")
```

```
jane()  
fred()  
jane()
```



FONCTIONS - PARAMÈTRES ET ARGUMENTS

- Un autre aspect fondamental des fonctions est qu'elles vont pouvoir accepter des informations qui viennent de l'extérieur.
- A l'intérieur de la fonction, les arguments sont assignés à des variables appelées paramètres.

Voici un exemple de fonction définie par l'utilisateur qui prend un argument:

```
def print_twice(something):  
    print(something)  
    print(something)  
  
print_twice("Spam")  
  
message = "Spam"  
print_twice(message)
```

- Possible d'utiliser n'importe quel type d'expression comme argument pour print_twice:

```
print_twice("Spam " * 4)
```



FONCTION AVEC PLUSIEURS PARAMÈTRES

- Pour définir une fonction avec plusieurs paramètres, il suffit d'inclure ceux-ci entre les parenthèses qui suivent le nom de la fonction, en les séparant à l'aide de virgules.
- Lors de l'appel de la fonction, les arguments utilisés doivent être fournis dans le même ordre que celui des paramètres correspondants.
- Cependant, il est possible d'indiquer lors de l'appel de la fonction à quels paramètres on veut envoyer nos arguments.

```
def politesse(nom, prenom, titre):  
    print("Veuillez agréer,", titre, nom, prenom, ", mes salutations distinguées.")
```

```
politesse("Lemuel", "Gulliver", "Mr")
```

```
politesse(prenom = "Lemuel", titre = "Mr", nom = "Gulliver")
```



VALEURS PAR DÉFAUT POUR LES PARAMÈTRES

- Lorsqu'on définit une fonction `def politesse(nom, prenom, titre)` : les arguments **nom, prenom et titre** sont appelés **arguments positionnels** (positional arguments). Il est strictement obligatoire de les préciser lors de l'appel de la fonction.
- Mais il est aussi possible de passer un ou plusieurs argument(s) de manière facultative et de leur attribuer une valeur par défaut (**arguments par mot clé**). On obtient ainsi une fonction qui peut être appelée avec une partie seulement des arguments attendus.

```
def politesse(nom, prenom, titre = "Mr"):  
    print("Veuillez agréer,", titre, nom, prenom,  
          ", mes salutations distinguées.")  
  
politesse("Lemuel", "Gulliver")
```



PORTÉE DES VARIABLES

- Les variables créées au sein d'une fonction ne sont pas visibles à l'extérieur de celle-ci car elles sont **locales** à la fonction.

```
def ma_fonction():  
    x = 2  
    print(f"x vaut {x} dans la fonction")
```

```
ma_fonction() —————> x vaut 2 dans la fonction  
print(x) —————> NameError: name 'x' is not defined
```



PORTÉE DES VARIABLES

- Une variable déclarée dans le programme principal est visible dans celui-ci ainsi que dans toutes les fonctions. Il s'agit d'une variable **globale**:

```
def ma_fonction():  
    print(x)  
x = 3  
ma_fonction()  
print(x)
```

- Toutefois, Python ne permet pas la modification d'une variable globale dans une fonction:

```
def ma_fonction():  
    x = x + 1  
x = 1
```

```
ma_fonction() → UnboundLocalError: local variable 'x' referenced before  
assignment
```



PORTÉE DES VARIABLES

- Si on veut vraiment modifier une variable globale dans une fonction, il faut utiliser le mot-clé **global**:

```
def ma_fonction():  
    global x  
    x = x + 1  
x = 1  
ma_fonction()  
print(x) → 2
```



PORTÉE DES VARIABLES

- Prédisez le comportement des codes suivants, sans les recopier dans un script ni dans l'interpréteur Python :

1

```
def hello(prenom):  
    print(f"Bonjour {prenom}")  
    print(x)
```

```
# Programme principal.  
x = 10  
hello("Flimnap")  
print(x)
```

2

```
def hello(prenom):  
    x = 42  
    print(f"Bonjour {prenom}")  
    print(x)
```

```
# Programme principal.  
x = 10  
hello("Patrick")  
print(x)
```



« VRAIES » FONCTIONS ET PROCÉDURES

- Les fonctions comme `print_twice` qui effectuent une action mais ne renvoient pas de valeur sont appelés des procédures ou fonctions void « **void functions** ».
- Une « vraie » fonction (au sens strict) doit en effet **renvoyer une valeur** lorsqu'elle se termine. Exemple:

```
def cube(w):  
    return w * w * w
```

- L'instruction `return` définit ce que doit être la valeur renvoyée par la fonction. En l'occurrence, il s'agit du cube de l'argument qui a été transmis lors de l'appel de la fonction.

```
b = cube(9)  
print(b)
```



FONCTIONS - EXERCICE

Écrire une fonction maximum permet d'afficher le plus grand de trois nombres entrés au clavier.

Solution 1

```
def maximum(a, b, c):  
    max = 0  
    if a > b:  
        max = a  
    else:  
        max = b  
    if max < c:  
        max = c  
    return max  
  
a = int(input("Type a value of the number a "))  
b = int(input("Type a value of the number b "))  
c = int(input("Type a value of the number c "))  
print(maximum(a, b, c))
```



FONCTIONS - EXERCICE

Solution 2:

```
def max_de_deux(x, y):  
    if x > y:  
        return x  
    return y  
def maximum(x, y, z):  
    return max_de_deux(x, max_de_deux(y, z))  
print(maximum(int(input("Saisir le 1er nombre")),int(input("Saisir le  
2eme nombre")),int(input("Saisir le 3eme nombre"))))
```



MODULES

- Module: tout fichier constitué de code Python importé dans un autre fichier ou script.
- Les modules permettent la séparation du code, ce qui favorise une meilleure organisation, une facilité de réutilisation, une meilleure lisibilité, un débogage efficace et un travail en équipe plus fluide
- Un module peut contenir des fonctions, des classes, des variables ou d'autres éléments de code.
- Quelques modules Python standards à connaître: `datetime`, `math`, `random`, `csv`, `sqlite3`...
- Pour la liste complète tapez la commande: `help("modules")`



MODULES

- Le module **math** fournit ensemble de constantes et de fonctions pour effectuer la plupart des opérations mathématiques familières:
 - **sqrt(x)** : Donne la racine carrée de x.
 - **cos(x), sin(x) et tan(x)** : Donnent respectivement le cosinus, le sinus et la tangente de x lorsque x est en radians
 - **radians(x)** : Convertit l'angle x des degrés en radians.
 - **acos(x), asin(x), atan(x)** : Donnent respectivement l'arccos, l'arcsin et l'arctan de x en radians.
 - **exp(x), log(x)** : Donne l'exponentielle et le logarithme népérien de x. Si on veut le logarithme en base b, il suffit de taper **log(x,b)**.
 - **pow(x,y)** : Renvoie x élevé à la puissance y
 - **pi** : Donne la valeur de pi avec la meilleure précision disponible.
- Le module **random** fournit des fonctions qui génèrent des nombres pseudo-aléatoires: **random(), randint(low,high), choice(object)**...



MODULES

➤ Syntaxes d'importation

- 1) `import nom_module`
OU
- 2) `from nom_module import nom_membre`
OU
- 3) `from nom_module import *`

➤ Syntaxes d'accès à un élément d'un module importé

- 1) `nom_module.nom_membre`
OU
- 2) `nom_membre` si l'importation est faite avec 2) ou 3)

